LAPORAN PRAKTIKUM MIKROPROSESOR DAN INTERFACE

KONVERSI BILANGAN HEKSADDESIMAL KE DESIMAL

Disusun untuk memenuhi tugas praktikum Mikroprosesor Dan Interface

Oleh

Gaitsha Azzahra Devitrya

231344012



Tanggal Percobaan: 20 Mei 2025

Tanggal Penyerahan: 27 Mei 2025

Instruktur: 1. Ferry Satria, BSEE., M.T.

2. Rahmawati Hasanah, S.ST., M.T.

PROGRAM STUDI D4 TEKNIK TELEKOMUNIKASI

JURUSAN TEKNIK ELEKTRO

POLITEKNIK NEGERI BANDUNG

2025

DAFTAR ISI

DAFTAR ISI ii
A. TUJUAN 2
B. LANDASAN TEORI 2
C. ALAT DAN BAHAN 4
D. LANGKAH KERJA 5
E. HASIL PERCOBAAN DAN ANALISIS DATA 6
G. KESIMPULAN 16
DAFTAR PUSTAKA 17

A. TUJUAN

1. Mahasiswa dapat memahami konsep konversi bilangan heksadesimal ke desimal.
2. Mahasiswa dapat membuat program pada EMU8086 untuk menyelesaikan konversi bilangan heksadesimal ke desimal.

B. LANDASAN TEORI

A. Definisi microprosesor

Mikroprosesor dikenal juga sebagai CPU (Central Processing Unit). CPU terdiri dari ALU, Control Unit, Register , Bus System, dan Clock untuk melakukan komputasi.

B. Register

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Register merupakan sebagian memori dari mikroprosesor yang dapat diakses dengan kecepatan yang sangat tinggi. Dalam melakukan pekerjaannya mikroprosesor selalu menggunakan register-register sebagai perantaranya, jadi register dapat diibaratkan sebagai kaki dan tangannya mikroprosesor.

Terdapat dua jenis register yang dibedakan menurut besar Bit, yaitu Register 8-Bit dan Register 16-Bit.

Register 8-Bit hanya memiliki 1 bagian jenis register yaitu Register umum : AH, AL, BH, BL, CH, CL, DH, DL

Dan Register 16-Bit memiliki 5 bagian jenis register yang memiliki tugasnya masing-masing, yaitu :

1. Segmen Register.

   Register yang termasuk dalam kelompok ini terdiri atas register CS (Code Segment), DS (Data Segment), ES (Extra Segment) dan SS (Stack Segment) yang masing-masingnya merupakan register 16 bit. Register-register dalam kelompok ini secara umum digunakan untuk menunjukkan alamat dari suatu segmen.

2. General Register (Register Umum)

   Register yang termasuk dalam kelompok ini adalah register AX, BX, CX dan DX yang masing-masing terdiri atas 16 bit. Register- register 16 bit dari kelompok ini mempunyai suatu ciri khas, yaitu dapat dipisah menjadi 2 bagian dimana masing-masing bagian terdiri atas 8 bit, Akhiran H menunjukkan High sedangkan akhiran L menunjukkan Low.

3. Register Pointer

Register ini menunjukkan alamat sebuah data di lokasi memori, dipakai saat operasi perpindahan data(dari/ke memori) dan penunjukkan alamat suatu instruksi. Jenis-jenis register pointer yaitu BP (Base Pointer) dan IP (Instruction Pointer).

4. Register Index

Sama dengan pointer register, sering digunakan untuk menunjukkan alamat sebuah data di lokasi memori pada operasi string. Jenisnya ada SI (Source Index), DI (Destination Index).

5. Register Stack

Register Stack (SP) atau Stack Pointer adalah salah satu jenis register dalam arsitektur komputer yang digunakan untuk menunjuk alamat teratas dari stack dalam memori.

6. Register Flag [REG 16 BIT]

Register Flag berfungsi untuk menunjukkan status(keadaan) sesaat dari mikroprosesor. Bit-bit pada flag akan mengalami perubahan, tergantung proses yang berlangsung. Sesuai dengan namanya Flags(Bendera) register ini menunjukkan kondisi dari suatu keadaan< ya atau tidak >. Karena setiap keadaan dapat digunakan 1 bit saja, maka sesuai dengan jumlah bitnya, Flags register ini mampu memcatat sampai 16 keadaan.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
-	-	-	-	OF	DF	IF	TF	SF	ZF	-	AF	-	PF	-	CF

C. Syntax/Perintah Mikroprosesor 8086

1. ADC: Penjumlahan dengan Carry
2. ADD: Penjumlahan
3. CMP : Membandingkan
   Memiliki Algoritma: Operand 1 – Operand 2
   Hasil tidak disimpan dimana-mana, hanya saja mempengaruhi Flag (OF, SF,ZF, AF, PF, CF) berdasarkan hasil.
4. MOV : Instruksi MOV tidak bisa dilaksanakan jika:
   - Memberikan nilai ke Register CS dan Register IP
   - Tidak bisa Menyalin nilai dari satu segmen ke segmen Register yang lain (Seharusnya menggunakan Register umum terlebih dahulu)
   - Menyalin segera data ke segmen register (Seharusnya menggunakan Register umum terlebih dahulu)
5. INC : Menaikkan

Memiliki Algoritma : Operand = Operand + 1

6. JNZ(Jump if Not Zero) = untuk melakukan lompatan ke alamat tertentu jika hasil operasi sebelumnya tidak nol.
7. DUP : untuk pendeklarasian variabel di segmen .DATA untuk mengalokasikan memori dalam jumlah tertentu dengan nilai awal yang sama.
8. DAT bukan instruksi bawaan Assembly, yang digunakan adalah DB (Define Byte), DW (Define Word), atau DD (Define Double Word) untuk mendefinisikan data.
   - DB (Define Byte) → Menyimpan 1 byte data
   - DW (Define Word) → Menyimpan 2 byte data
   - DD (Define Double Word) → Menyimpan 4 byte data

D. Perintah DAA (Decimal Adjust after Addition)

Perintah DAA (Decimal Adjust after Addition) digunakan untuk menyesuaikan hasil penjumlahan agar tetap dalam format BCD (Binary-Coded Decimal) setelah operasi penambahan. Instruksi ini bekerja dengan memeriksa nilai di register AL setelah operasi penjumlahan biasa menggunakan ADD atau ADC**.** Jika bagian rendah dari register AL (4 bit paling rendah) lebih besar dari 9 atau jika AF (Auxiliary Carry Flag) diatur, maka mikroprosesor akan menambahkan 6 (06H) ke bagian rendah AL untuk memastikan hasil tetap dalam format BCD. Demikian pula, jika bagian tinggi AL (4 bit paling atas) lebih besar dari 9 atau jika CF (Carry Flag) diatur, maka mikroprosesor akan menambahkan 60H ke AL dan mengatur CF sebagai tanda bahwa ada carry ke digit berikutnya. Perintah DAA umumnya digunakan dalam aplikasi yang menangani angka desimal.

E. Konversi Bilangan Heksadesimal ke Desimal

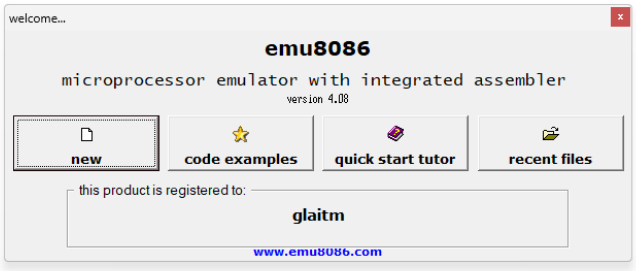
Konversi bilangan heksadesimal ke desimal merupakan proses mengubah angka dari sistem bilangan berbasis 16 ke dalam sistem bilangan berbasis 10. Sistem heksadesimal menggunakan 16 simbol, yaitu angka 0 hingga 9 dan huruf A hingga F, di mana huruf A sampai F mewakili nilai desimal 10 hingga 15. Proses konversi dilakukan dengan cara mengalikan setiap digit heksadesimal dengan 16 pangkat posisi digit tersebut, dihitung dari kanan ke kiri dimulai dari posisi nol. Misalnya, bilangan heksadesimal "2F" dikonversi menjadi desimal dengan cara menghitung 2×161+15×160=32+15=472 \times 16^1 + 15 \times 16^0 = 32 + 15 = 47. Sistem bilangan heksadesimal banyak digunakan dalam dunia komputer, terutama dalam pemrograman dan pengalamatan memori, karena lebih ringkas dibandingkan bilangan biner dan mudah dikonversi ke sistem lain.

C. ALAT DAN BAHAN

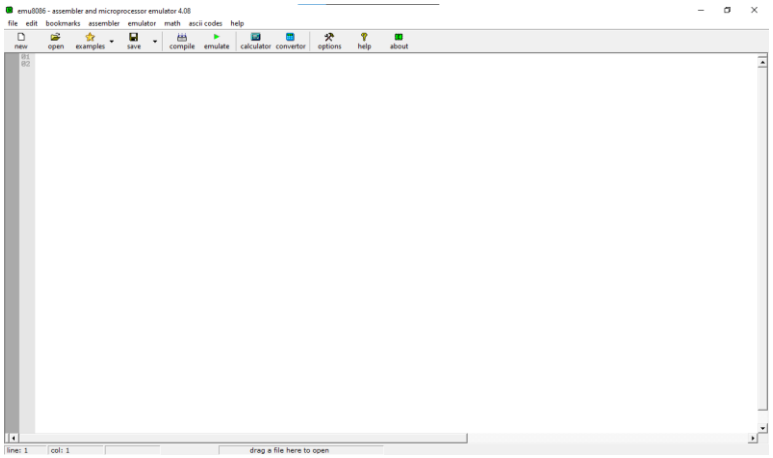
1. Laptop/PC
2. Software EMU 8086

D. LANGKAH KERJA

1. Instal Software emu8086 pada laptop dan buka aplikasinya
2. Setelah membuka aplikasi maka akan muncul tampilan *new* pada monitor

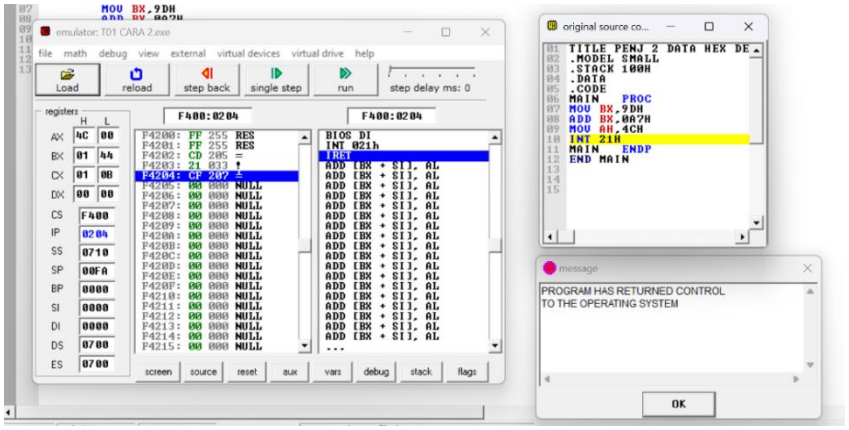


3. Maka setelah itu akan tampil worksheet seperti gambar dibawah ini :



Worksheet ini digunakan untuk menuliskan kode.

4. Untuk memeriksa kode yang berada di worksheet apakah program sudah benar atau belum, klik “compile”
5. Jika sudah disave maka jika program sudah benar akan ada tulisan “run”.
6. Jika sudah di ”run” maka akan muncul hasilnya pada monitor



E. HASIL PERCOBAAN DAN ANALISIS DATA

Pada praktikum ini, dilakukan praktikum untuk membuat program assembly pada software EMU8086 untuk mengonversi atau mengubah data bilangan heksadesimal ke desimal. Dengan terdapat dua metode konversi yaitu perkalian berulang dan metode look up table yang diimplementasikan pada pembuatan program assembly di software EMU8086.

Pada praktikum kali ini terdapat 5 soal yang berhasil dikerjakan yang dapat dianalisis sebagai berikut :

1. Buat program untuk mengubah data heksadesimal 2 digit yang ada pada register CH menjadi desimal. Simpan hasil konversi pada register DX.

   Program :

   Cara 1 : Perkalian Berulang

```
TITLE SOAL 1 ;CARA PERKALIAN BERULANG
.MODEL SMALL
.STACK 100H
.DATA
.CODE
 MAIN PROC
      MOV BH,CH
      AND BH,0FH
      MOV AL,BH
      ADD AL,0
      DAA
      MOV BH,AL

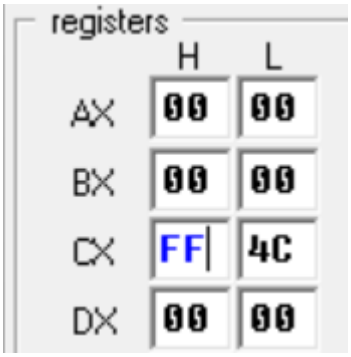
      MOV BL,CH
      AND BL,0F0H
      SHR BL,4
      MOV AL,BL
      ADD AL,0
      DAA
      MOV BL,AL
      MOV DX,0
      MOV CL,4
      ADD DL,BL

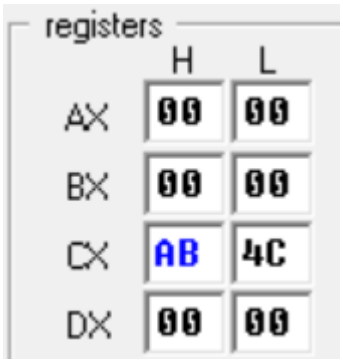
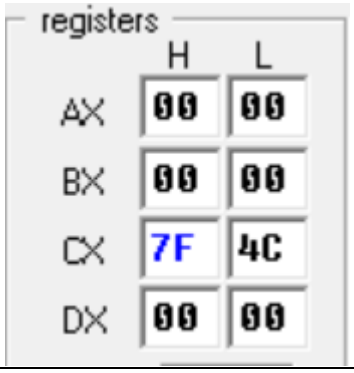
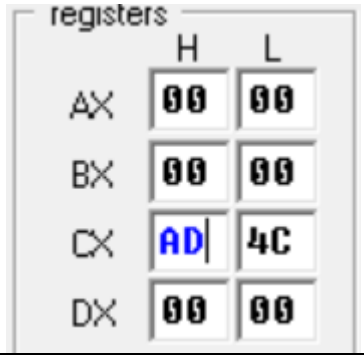
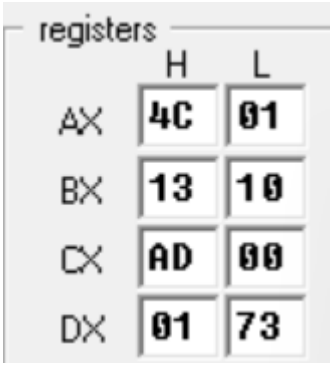
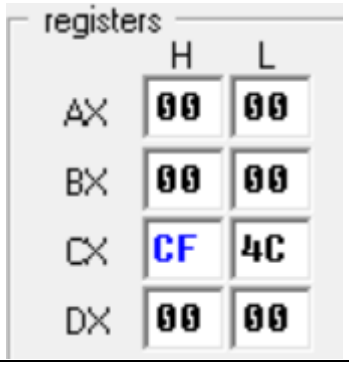
LUP:  MOV AL,DL
      ADD AL,DL
      DAA
      MOV DL,AL
      MOV AL,DH
      ADC AL,DH
      DAA
      MOV DH,AL
      SUB CL,1

      JNZ LUP

      MOV AL,DL
      ADD AL,BH
      DAA
      MOV DL,AL
      MOV AL,DH
      ADC AL,0
      DAA
      MOV DH,AL
      MOV AH,4CH
      INT 21H
MAIN  ENDP
      END MAIN
```

Hasil program:

Data Percobaan	Hasil Percobaan
FF = 255 	

AB = 171 	
7F= 127 	
AD = 173 	
CF = 207 	

Cara 2: Look up table

Program :

```
TITLE SOAL 2 ;CARA LOOK UP TABLE
.MODEL SMALL
.STACK 100H
.DATA
 TABEL1 DB 0H,1H,2H,3H,4H,5H,6H,7H,8H,9H,10H,11H,12H,13H,14H,15H
 TABEL2 DW 0H,16H,32H,48H,64H,80H,96H,112H,128H,144H,160H,176H,192H,208H,224H,240H
.CODE
 MAIN PROC
      MOV AX,@DATA
      MOV DS,AX

      MOV CH,0F6H
      MOV BL,CH
      AND BL,0FH

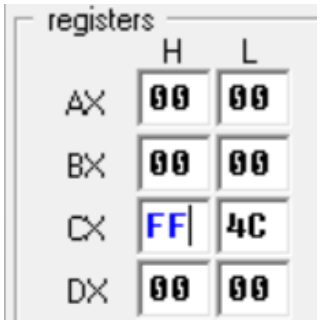
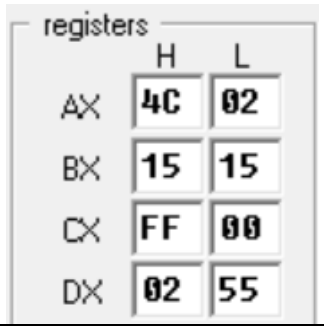
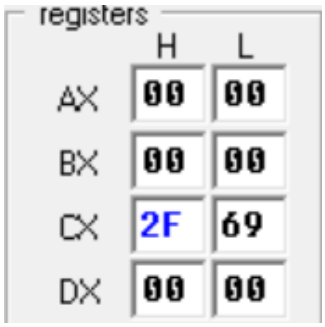
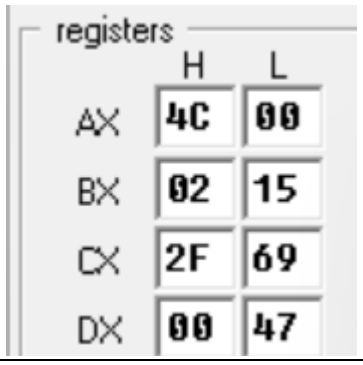
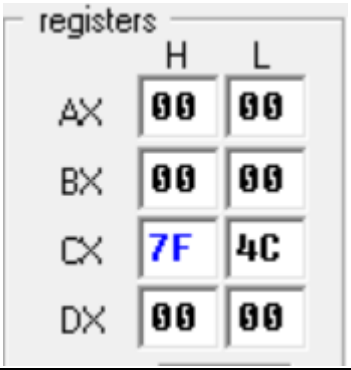
      MOV AL,BL
      MOV AH,0
      MOV SI,AX
      MOV BL,[TABEL1 + SI]

      MOV BH,CH
      AND BH,0F0H
      SHR BH,4

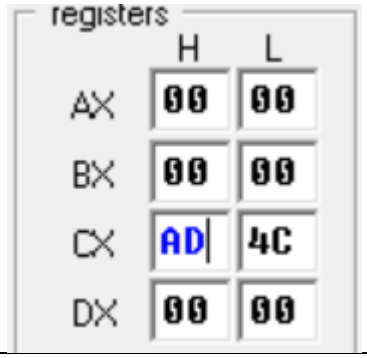
      MOV DL,BH
      MOV DH,0
      ADD DX,DX
      MOV SI,DX

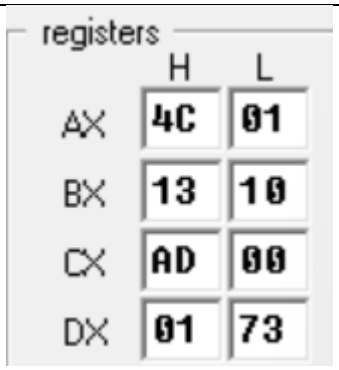
      MOV DX,[TABEL2+SI]
      MOV AL,DL
      ADD DL,BL
      MOV AL,DH
      ADC AL,0
      DAA


      MOV AH,4CH
      INT 21H
 MAIN ENDP
      END MAIN
```

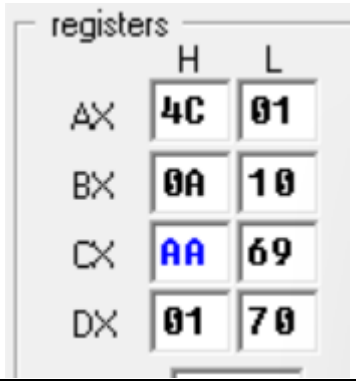
Data Percobaan	Hasil Percobaan
FF = 255 	
2F = 246 	
7F= 127 	

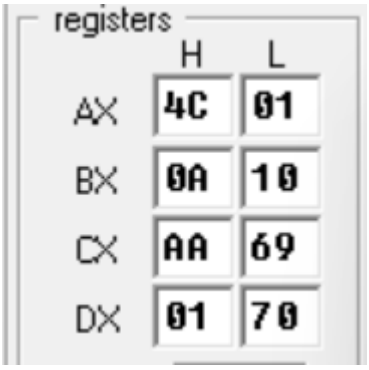
AD = 173





AA = 170





Analisis Data :

Pada percobaan pertama, dilakukan pembuatan program untuk mengonversi bilangan heksadesimal 2 digit ke bilangan desimal menggunakan bahasa assembly pada EMU8086. Input data pada kedua metode dimasukkan melalui register CH, dan hasil konversi ditampilkan melalui register DX.

Pada program pertama, konversi dilakukan dengan metode matematika manual, yakni memisahkan digit tinggi (nibble atas) dan digit rendah (nibble bawah) dari register CH. Digit tinggi diambil dengan AND BH, 0F0H lalu digeser ke kanan sebanyak 4 bit (SHR BH, 4) untuk mendapatkan nilai digit pertama, sedangkan digit rendah diperoleh dengan AND BL, 0FH. Kedua digit heksadesimal (masing-masing 0–15) kemudian dikonversi ke bentuk desimal melalui perkalian manual, yakni dengan menjumlahkan nilai digit tinggi sebanyak 10 kali, menggunakan pengulangan LUP: untuk mensimulasikan operasi digit_atas * 10. Setiap hasil penjumlahan memperhatikan carry dari register sebelumnya menggunakan instruksi DAA untuk penyesuaian desimal. Nilai akhir dari konversi dimasukkan ke dalam register DX.

Kemudian, pada program kedua menggunakan metode mengakses data hasil konversi yang telah disiapkan dalam tabel. Dua tabel digunakan: TABEL1 berisi nilai-nilai desimal dari digit heksadesimal 0–15 dalam bentuk byte, dan TABEL2 berisi hasil digit heksadesimal dikali 16 (yaitu 0, 16, 32, ..., 240) dalam bentuk word. Seperti pada program pertama, nilai CH dipisah menjadi digit atas (CH AND 0F0H → SHR) dan digit bawah (CH AND 0FH). Kedua nilai tersebut dijadikan indeks ke dalam TABEL1 dan TABEL2. Hasilnya

diambil langsung dari memori menggunakan perintah seperti MOV BL, [TABEL1 + SI] dan MOV DX, [TABEL2 + SI]. Setelah itu, hasil akhir ditambahkan dan disesuaikan kembali menggunakan instruksi DAA.

Metode pertama lebih manual dan lebih lambat karena menggunakan loop untuk mengalikan dan menjumlahkan. Sementara itu, metode kedua lebih cepat karena menggunakan look-up table yang sudah berisi hasil konversi dan waktu proses cepat. Kedua metode menampilkan hasil akhir pada register DX.

2.Buat Program untuk mengubah data heksadesimal 4 digit yang ada pada register BX menjadi desimal. Simpan hasil konversi pada register CX:DX

Program :

Cara 1 : Perkalian Berulang

```
TITLE SOAL 2 4 DIGIT KONVERSI ;CARA 1 PERKALIAN BERULANG
.MODEL SMALL
.STACK 100H
.DATA
    CARRY DW 0
.CODE
MAIN    PROC
        MOV AX, @DATA
        MOV DS, AX

        MOV AH, 0
        MOV CX, 0
        MOV DX, 0
        MOV [CARRY], BX
        MOV BX, 0

L1:     MOV AL, DL
        ADD AL, 1
        DAA
        MOV DL, AL

        MOV AL, DH
        ADC AL, 0
        DAA
        MOV DH, AL

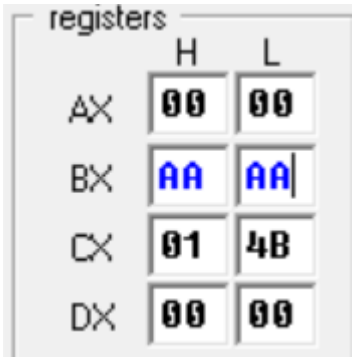
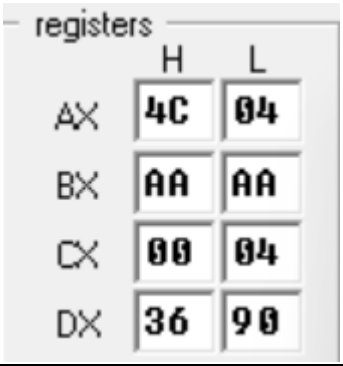
        MOV AL, CL
        ADC AL, 0
        DAA
        MOV CL, AL
        MOV CH, 0


        ADD BL, 1
        ADC BH, 0
        CMP BX, [CARRY]
        JNZ L1


        MOV AH,4CH
        INT 21H
MAIN    ENDP

        END MAIN
```

Hasil program:

Data Percobaan	Hasil Percobaan
AAAA = 43690 	

Cara 2 : look up table

```
TITLE SOAL3 ;KONVERSI 4 DIGIT CARA LOOK UP TABLE
.MODEL SMALL
.STACK 100H
.DATA
    TBL1 DB 0H, 1H, 2H, 3H, 4H, 5H, 6H, 7H, 8H
         DB 9H, 10H, 11H, 12H, 13H, 14H, 15H

    TBL2 DW 0, 16H, 32H, 48H, 64H, 80H, 96H, 112H, 128H
         DW 144H, 160H, 176H, 192H, 208H, 224H, 240H

    TBL3 DW 0, 256H, 512H, 768H, 1024H, 1280H, 1536H, 1792H
         DW 2048H, 2304H, 2560H, 2816H, 3072H, 3328H, 3584H
         DW 3840H

    TBL4 DW 0, 0, 4096H, 0, 8192H, 0, 2288H, 1H, 6384H, 1H
         DW 0480H, 2H, 4576H, 2H, 8672H, 2H, 2768H, 3H
         DW 6864H, 3H, 0960H, 4H, 5056H, 4H, 9152H, 4H
         DW 3248H, 5H, 7344H, 5H, 1440H, 6H
    CARRY DB 0,0,0,0,0
.CODE
MAIN    PROC
        MOV AX, @DATA
        MOV DS, AX

        MOV DI, BX

        MOV BX, DI
        MOV BL, BH
        AND BL, 0FH
        MOV AL, BL
        MOV AH, 0
        MOV SI, AX
        MOV BL, [TBL1 + SI]

        MOV BX, DI
        MOV BL, BH
        AND BL, 0F0H
        SHR BL, 4

        MOV DL, BL
        MOV DH, 0
        ADD DX, DX
        MOV SI, DX
        MOV DX, [TBL2 + SI]
        ADD DL, AL
        DAA
        MOV AL, 0
        ADC DH, AL
        DAA

        MOV BX, DI
        AND BH, 0FH
        MOV AL, BH
        MOV AH, 0
        MOV SI, AX
        ADD SI, SI
        MOV BX, [TBL3 + SI]

        MOV [CARRY], BL
        MOV AL, [CARRY]
        ADD AL, DL
        DAA
        MOV DL, AL

        MOV [CARRY + 1], BH
        MOV AL, [CARRY + 1]
        ADC AL, DH
        DAA
        MOV DH, AL

        MOV BX, DI
        AND BH, 0F0H
        SHR BH, 4

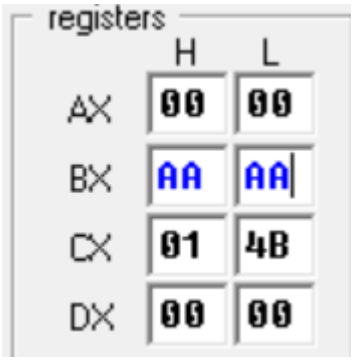
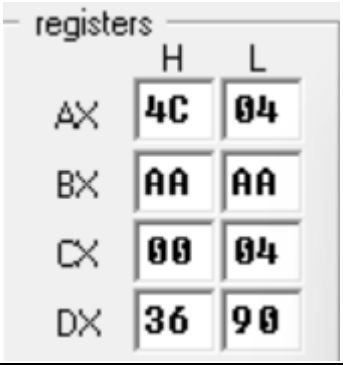
        MOV CL, BH
        MOV CH, 0
        MOV SI, CX
        ADD SI, SI
        ADD SI, SI
        MOV CX, [TBL4 + SI]
        MOV [CARRY + 2], CL
        MOV [CARRY + 3], CH
        MOV AX, [TBL4 + SI + 1]
        MOV [CARRY + 4], AH

        MOV AL, [CARRY + 2]
        ADD AL, DL
        DAA
        MOV DL, AL
        MOV AL, [CARRY + 4]
        ADC AL, 0
        DAA
        MOV CL, AL
        MOV CH, 0

        MOV AH,4CH
        INT 21H
MAIN    ENDP

        END MAIN
```

Hasil program :

Data Percobaan	Hasil Percobaan
AAAA = 43690 	

Analisis Data :

Pada percobaan kedua ini, program dibuat untuk mengonversikan bilangan heksadesimal 4 digit ke bilangan desimal. Dengan menggunakan dua metode seperti sebelumnya, program yang dibuat hampir sama dengan percobaan pertama.

Pada program pertama, konversi dilakukan secara manual menggunakan perkalian berulang. Nilai heksadesimal dimasukkan ke register BX, lalu disalin ke CARRY. Kemudian dilakukan proses loop (L1:) sebanyak nilai DL (yang berasal dari BX) untuk melakukan operasi result = result + carry sebanyak nilai digit tersebut. Setiap kali penjumlahan dilakukan, digunakan instruksi DAA (Decimal Adjust after Addition) untuk mengubah hasil penjumlahan ke format BCD (Binary Coded Decimal), agar sesuai dengan bentuk desimal.

Untuk memperhitungkan digit lebih tinggi dari hasil perkalian, digunakan register BH untuk menyimpan hasil carry. Selanjutnya, CL yang awalnya bernilai 16 dipakai untuk menghitung pengulangan sebanyak 16 kali, menyesuaikan panjang digit 4-bit. Hasil akhir disimpan dalam register CX:DX, di mana nilai DX berasal dari hasil akhir penjumlahan dan CX dari carry.

Program kedua menggunakan metode look-up table yang menyimpan hasil dari berbagai kemungkinan nilai digit heksadesimal. Input tetap dimasukkan pada BX, namun input dimasukkan ke BX nya melalui program MOV BH, DATA lalu digit demi digit dari bilangan heksadesimal dipecah menggunakan instruksi AND dan SHR. Digit-digit ini digunakan sebagai indeks untuk mengambil nilai desimal yang sudah disiapkan dalam tabel TBL1, TBL2, TBL3, dan TBL4. Setiap tabel berisi hasil konversi dari kelipatan 1, 16, 256, dan 4096 (yaitu 16^0, 16^1, 16^2, 16^3) dalam bentuk desimal.

Nilai dari keempat tabel dijumlahkan menggunakan instruksi ADD dan ADC (Add with Carry) dan disesuaikan dengan instruksi DAA untuk memastikan format desimal yang benar. Penjumlahan dari keempat hasil ini disimpan berturut-turut dalam memori CARRY dan akhirnya disalin ke register DX dan CX.

3. Buat program untuk mengubah 16 data heksadesimal 2 digit yang ada pada BM1menjadi Desimal. Simpan hasil konversi pada BM2

Program :

```
TITLE SOAL 3 ;CARA PERKALIAN DIGIT
.MODEL SMALL
.STACK 100H
.DATA
BM1 DB 11H,22H,33H,44H,55H,66H,77H,88H,99H
    DB 1AH,2AH,3AH,4AH,5AH,6AH,7AH
BM2 DW 16 DUP(0)
.CODE
MAIN PROC
     MOV AX,@DATA
     MOV DS,AX
     MOV SI,0
     MOV BX,0
     MOV CX,0
     MOV DX,0

LUP1:MOV AX,0
     MOV CH,[BM1+SI]
     MOV AL,CH
     AND AL,0F0H
     MOV CL,4
     SHR AL,CL
     DAA

     CALL KONV
     MOV DX,AX
     MOV AL,CH
     AND AL,0FH
     DAA
     ADD AL,DL
     DAA

     ADC AH,0
     MOV [BM2+DI],AX
     INC SI
     INC DI
     INC DI
     CMP SI,16
     JNZ LUP1

     MOV AH,4CH
     INT 21H

MAIN ENDP

KONV PROC
     ADD AL,AL
     DAA

     ADD AL,AL
     DAA

     ADD AL,AL
     DAA

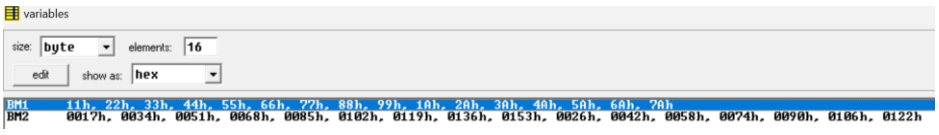
     ADC AH,0
     CMP AX,100H
     JAE LUP2
     ADD AL,AL
     DAA

     ADC AH,0
     RET

LUP2:ADD AX,AX
     DAA
     ADC AH,0
     RET

KONV ENDP
END MAIN
```

Hasil program:



Analisis Data :

Pada percobaan ketiga yaitu membuat program untuk mengonversi 16 data bilangan heksadesimal 2 digit yang tersimpan dalam array BM1 menjadi bilangan desimal, lalu menyimpan hasil konversinya ke dalam array BM2. Proses konversi dilakukan dengan cara memisahkan setiap digit dari bilangan heksadesimal digit (digit atas dan digit bawah) kemudian menghitung nilai desimalnya dengan mengalikan digit atas dengan 10 dan menambahkannya dengan digit bawah. Proses ini dilakukan dalam loop LUP1, yang akan menelusuri semua isi BM1. Untuk melakukan perkalian dengan 10, digunakan prosedur KONV yang secara berulang menjumlahkan nilai digit sebanyak 10 kali, namun dioptimalkan menggunakan instruksi ADD dan DAA untuk mempertahankan format desimal (BCD). Setelah nilai desimal diperoleh, hasilnya disimpan di array BM2 sebagai data 2 byte.

4.Buat program untuk mengubah 16 data heksadesimal 4 digit yang ada pada BM1 menjadi desimal. Simpan hasil konversi pada BM 2

Program :

```
TITLE SOAL 4 KONVERSI HEX-DES
.MODEL SMALL
.STACK 100H
.DATA
BM1 DW 1A3FH,  0AF12H,  3C4BH,  7FFFH,  8001H,  1234H,  0ABCDH,  0FFFFH
    DW 0AAAAH,  2468H,  0DEADH,  0BEEFH,  5A5AH,  0C0DEH,  0A55AH,  0ABADH
BM2 DW 32 DUP (0)
DAT1 DB 7 DUP (0)
DAT2 DB 7 DUP (0)
DAT3 DB 0
.CODE
MAIN PROC
     MOV AX, @DATA
     MOV DS, AX

     MOV [DAT3], 16H
     MOV DI, 0
     MOV SI, 0
LUP: MOV BX, 0
     MOV [DAT1], 0
     MOV [DAT1+1],0
     MOV [DAT1+2],0
     MOV [DAT1+3],0
     MOV [DAT1+4],0
     MOV [DAT1+5],0
     MOV [DAT1+6],0

     MOV BX, [BM1+SI]
     MOV DL, BL
     AND DL, 0FH
     MOV AL, DL
     ADD DL, 0
     DAA
     MOV DL, AL

     MOV AL, 0
     MOV DH, BL
     AND DH, 0F0H
     SHR DH, 4
     ADD AL, DH
     DAA
     MOV DH, AL

     MOV CL, BH
     AND CL, 0FH
     MOV AL, CL
     ADD AL, 0
     DAA
     MOV CL, AL

     MOV AL, 0
     MOV CH, BH
     AND CH, 0F0H
     SHR CH, 4
     ADD AL, CH
     DAA
     MOV CH, AL

     MOV AL, 0
     MOV BL, 0
     MOV BL, 4
     MOV [DAT1], DH
LUP: MOV AL, [DAT1]
     ADD AL, [DAT1]
     DAA
     MOV [DAT1], AL
     MOV AL, [DAT1+1]
     ADC AL, 0
     ADC [DAT1+1], AL
     DAA
     DEC BL
     JNZ LUP

     MOV AL, 0
     MOV BL, 0
     MOV BL, 8
     MOV [DAT1+2], CL
LUP1:MOV AL, [DAT1+2]
     ADD AL, [DAT1+2]
     DAA
     MOV [DAT1+2], AL
     MOV AL, [DAT1+3]
     ADC AL, 0
     DAA
     ADD AL, [DAT1+3]
     DAA
     MOV [DAT1+3], AL
     DEC BL
     JNZ LUP1

     MOV AL, 0
     MOV BL, 0
     MOV BL, 12
     MOV [DAT1+4], CH
LUP2:MOV AL, [DAT1+4]
     ADD AL, [DAT1+4]
     DAA
     MOV [DAT1+4], AL
     MOV AL, [DAT1+5]
     ADC AL, 0
     DAA
     ADD AL, [DAT1+5]
     DAA
     MOV [DAT1+5], AL
     MOV AL, [DAT1+6]
     ADC AL, 0
     DAA
     ADD AL, [DAT1+6]
     DAA
     MOV [DAT1+6], AL
     DEC BL
     JNZ LUP3

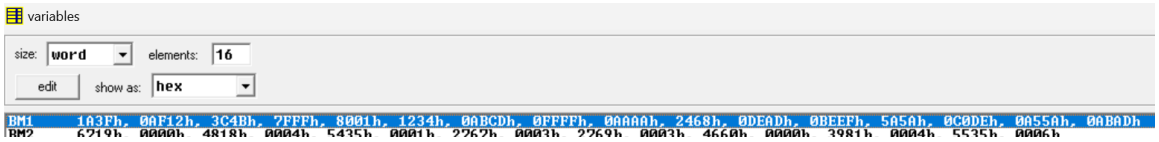
     MOV AL, DL
     ADD AL, [DAT1]
     DAA
     MOV [DAT2], AL
     MOV AL, [DAT1+1]
     ADC AL, 0
     DAA
     MOV [DAT2+1], AL

     MOV AL, [DAT2]
     ADD AL, [DAT1+2]
     DAA
     MOV [DAT2+2], AL
     MOV AL, [DAT2+1]
     ADC AL, 0
     DAA
     ADD AL, [DAT1+3]
     DAA
     MOV [DAT2+3], AL

     MOV AL, [DAT2+2]
     ADD AL, [DAT1+4]
     CALL SDAA
     MOV [DAT2+4], AL
     MOV AL, [DAT2+3]
```

```
137         MOV [DAT2+4], AL
138         MOV AL, [DAT2+3]
139         ADC AL, 0
140         CALL SDAA
141         ADD AL, [DAT1+5]
142         CALL SDAA
143         MOV [DAT2+5], AL
144         MOV AL, [DAT1+6]
145         ADC AL, 0
146         CALL SDAA
147         MOV [DAT2+6], AL
148
149         MOV DL,[DAT2+4]
150         MOV DH,[DAT2+5]
151         MOV CL,[DAT2+6]
152         MOV CH, 0
153         MOV [BM2+DI+2], CX
154         MOV [BM2+DI], DX
155
156         ADD SI, 2
157         ADD DI, 4
158         MOV AL, [DAT3]
159         SUB AL, 1
160         DAS
161         MOV [DAT3], AL
162         CMP [DAT3], 0
163         JNZ LUP
164         MOV AH, 4CH
165         INT 21H
166 MAIN ENDP
```

Hasil program:



Analisis Data :

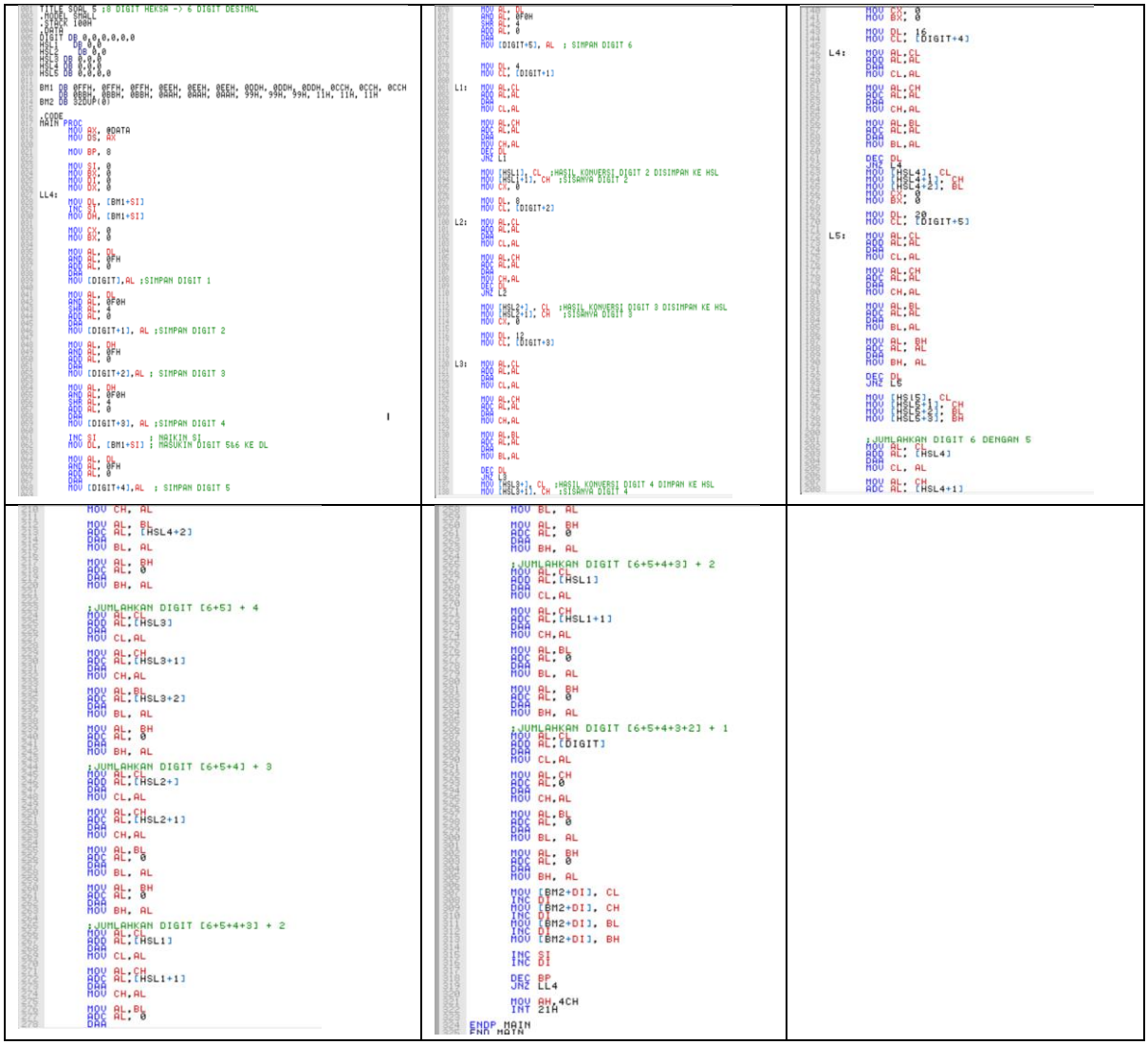
Pada percobaan keempat ini sama dengan percobaan ketiga dengan jumlah digit yang lebih besar yaitu 6 digit. Nilai yang akan di konversi disimpan di dalam array BM1 menjadi bilangan desimal, lalu menyimpan hasil konversinya ke array BM2. Proses konversi dilakukan secara manual dengan logika perhitungan dasar yaitu dengan perkalian berulang.

Pertama, program menginisialisasi segmen data dan menyimpan nilai 16 ke variabel DAT3 sebagai penghitung jumlah data yang akan diproses. Setiap bilangan heksadesimal yang terdiri dari 2 byte (16 bit atau 4 digit hex) dibaca satu per satu dari array BM1. Nilai tersebut kemudian dipecah menjadi digit-digit kecil berdasarkan posisi digit(4-bit), yang kemudian diolah untuk mendapatkan nilai desimalnya. Digit-digit hex tersebut dikalikan dengan pangkat 16 yang sesuai (1, 16, 256, 4096) dan diakumulasi ke buffer DAT1 dan DAT2 menggunakan teknik penjumlahan dan penanganan carry.

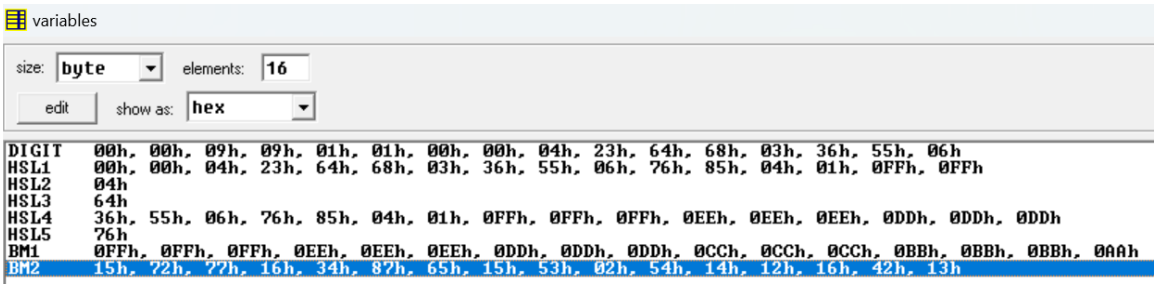
Setelah dilakukan pengolahan dan akumulasi, hasil akhir bilangan desimal akan tersimpan dalam buffer DAT2. Nilai dari DAT2 kemudian disalin ke array BM2, di mana setiap bilangan desimal direpresentasikan dalam format 4-byte (32-bit) untuk menampung angka hingga 65535. Proses ini diulang sebanyak 16 kali hingga seluruh data di BM1 dikonversi dan disimpan di BM2

5.Buat program untuk mengubah 8 data heksadesimal 6 digit menjadi desimal. Tentukan sendiri lokasi data input dan data output dalam unit memori (Data Segment Area)

Program :



Hasil Program :



Analisis Data :

Pada percobaan terakhir yaitu program digunakan untuk mengonversi data heksadesimal 6 digit (misalnya FFFFFF, C17799) menjadi format desimal. Program diawali dengan inisialisasi segmen data dan segmen kode. Dalam segmen data, terdapat delapan data heksadesimal yang akan dikonversi. Tiap nilai heksadesimal terdiri dari tiga byte dan disimpan berurutan dalam array.

Di bagian kode utama (MAIN), program mengambil masing-masing byte dari nilai heksadesimal dan menyimpannya ke dalam array. Proses ini dilakukan

dalam loop yang mengakses setiap dua digit dari memori dan memisahkannya menjadi digit individu yang bisa diolah satu per satu.

Setelah memecah digit-digit tersebut, proses konversi dilakukan dalam beberapa tahapan, yaitu dengan cara mengalikan setiap digit heksadesimal dengan bobot posisinya dalam sistem basis 16, kemudian menjumlahkan hasil-hasil perkalian tersebut. Ini dilakukan melalui beberapa label seperti L1, L2, L3, dan seterusnya. Perhitungan melibatkan operasi rotasi dan penjumlahan menggunakan register seperti AL, BL, CL, dan BH.

Contoh: Digit ke-6 dikalikan dengan 1, digit ke-5 dikalikan dengan 16, digit ke-4 dengan 16^2 (256), dan seterusnya. Ini sesuai dengan prinsip konversi dari heksadesimal ke desimal, yaitu:
Nilai = $D_5 \times 16^5 + D_4 \times 16^4 + ... + D_0 \times 16^0$.

Hasil penjumlahan akhir dari digit-digit yang telah dikalikan akan disimpan ke array hasil (HSL) yang nantinya akan menjadi hasil akhir dalam bentuk desimal. Data hasil tersebut kemudian ditampilkan atau diproses lebih lanjut sesuai kebutuhan.

Hasilnya pada array BM2 dengan ditampilkan dalam per 2 digit, hasil konversi sudah benar seperti contoh :



Artinya program berhasil mengonversikan nilai 6 digit heksadesimal ke bilangan desimal yang hasilnya di tampilkan pada array BM2.

F. KESIMPULAN

Pada praktikum ini dapat disimpulkan bahwa praktikan dapat lebih memahami cara kerja software EMU8086 yang dapat menjalankan program yang digunakan untuk mikroprosesor. Praktikan juga lebih memahami bagaimana cara membuat program mengonversi bilangan dari heksadesimal ke desimal. Terdapat 3 metode yang dipelajari, yaitu metode perkalian berulang, look up table dan cara alternatif. Namun pada praktikum ini, praktikan hanya mengimplementasikan 2 cara yaitu look up table dan perkalian berulang yang dimana kedua metode tersebut berhasil diimplementasikan sehingga praktikan berhasil membuat program untuk mengonversi bilangan heksadesimal ke desimal yang akhirnya ditampilkan bisa di register maupun di vars atau array.

DAFTAR PUSTAKA

Arief Wisaksono, I. M. (2019). *Buku Ajar Mikroprosesor* . Retrieved from eprint.umsida.ac.id:http://eprints.umsida.ac.id/7199/1/31.%20buku%20ajar%20mikroprosessor%20komplate%20-%20Arief%20Wisaksono.pdf

Hasanah, Rahmawati. 2019. Mikroprosesor dan Interface. Bandung: Politeknik Negeri Bandung

S.to. 2001. “Pemrograman Dengan Bahasa Assembly Edisi Online Versi 1.0